Real-Time Cinematic Tracking Gimbal: Edge-Computed Pose Estimation and Symmetric Kinematic Control

Yining Liu (3153782Y) | Zongwei Xie (3085969X) | Yifei Wang (3147822W) | Yandong Fan (3159430F) | Chenxu Li (3091645L)

1. Project Overview

1.1 Background and Motivation

The rapid proliferation of solo content creation, dynamic live streaming, and fast-paced sports videography has fundamentally shifted the requirements for camera operation. Traditionally, capturing moving subjects with high cinematic quality required a dedicated, skilled human camera operator to anticipate movement and manually adjust the camera's pan and tilt. Static tripod setups, conversely, heavily restrict the subject's freedom of movement and often result in subjects moving out of frame.

This creates a critical demand for autonomous, edge-computing-based vision-tracking gimbals. Such systems act as intelligent, robotic camera operators capable of maintaining the subject within the optimal compositional frame without human intervention. By deploying advanced Computer Vision (CV) algorithms directly on edge hardware, a tracking gimbal can continuously analyze the scene, estimate the subject's spatial coordinates, and drive closed-loop motor control to execute fluid tracking movements. This project addresses the gap between computationally expensive, server-reliant tracking systems and low-end, unresponsive consumer gimbals by developing a highly responsive, standalone dual-axis tracking solution.

1.2 Core Objectives

The primary aim of this project is to engineer an autonomous dual-axis tracking gimbal system that bridges advanced AI perception with precision motor control. To achieve a cinematic and robust tracking experience, the system is designed around the following core engineering objectives:

High Real-Time Performance (Low Latency): The perception-to-actuation loop must operate with minimal delay to prevent the target from escaping the field of view during rapid lateral or depth movements. This requires optimizing lightweight neural networks (e.g., YOLOv8-Pose) for deployment on edge devices (Raspberry Pi 5) using efficient inference frameworks (NCNN), ensuring the system processes frames and dispatches velocity commands at a consistently high refresh rate.

Superior Kinematic Smoothness (Elimination of Reversal Jerks): A common flaw in rudimentary tracking gimbals is mechanical stuttering, particularly during target deceleration or directional changes. A core objective is to design a sophisticated kinematic planner utilizing symmetric linear ramping and low-pass filtering. This ensures that motor acceleration and deceleration remain uniform, effectively eliminating high-frequency oscillations (sawtooth waves) and directional switching jerks (stuttering), thereby yielding

professional, silken camera movements.

Robust Dynamic Target Locking (Solving Compositional Drift): Traditional bounding-box-based tracking often suffers from "center-of-mass drift"—where the camera tilts erratically when a subject approaches the lens and their lower body is cropped out of frame. This project aims to solve this physical paradox by upgrading the perception engine to skeletal keypoint estimation (Pose Tracking). By locking onto specific anatomical anchors (e.g., the subject's nose or shoulders), the gimbal maintains rock-solid vertical composition regardless of the subject's distance or partial occlusion.

Seamless Human-Robot Interaction (HRI):

To maximize usability in solo-operator scenarios, the system aims to incorporate non-contact spatial interaction. By leveraging the same skeletal keypoint data used for tracking, the system will recognize specific human gestures (e.g., sustained wrist elevation) to trigger state machine transitions (pausing or resuming tracking) and provide physical kinematic feedback (e.g., a "nodding" motion), ensuring intuitive and remote operational control.

2. Overall System Hardware and Software Architecture

To ensure smoothness and rapid response during the dynamic tracking process of the gimbal, the overall system is divided from top to bottom into three highly independent yet closely collaborative layers.

2.1 Vision and Decision Layer

This layer uses Raspberry Pi as the core hardware carrier, is equipped with a non-real-time operating system, and assumes full responsibility for the macro decision-making tasks with the highest computing density in the system. Its primary physical reliance is the Raspberry Camera 3. The system can strip away the background environment in real-time at a fixed video frame rate and continuously lock onto the screen coordinates of the dynamic target. After obtaining the pixel-level spatial offsets, the PID control logic will intervene, smoothly converting the messy coordinate errors into the desired movement speeds for both the horizontal and pitch axes of the gimbal.

2.2 Hard Real-time Drive Layer

This layer takes the Raspberry Pi Pico 2 microcontroller as its core. Unlike the Raspberry Pi, the microcontroller is detached from the task scheduling mechanism of the operating system and focuses on extremely strict microsecond-level hard real-time tasks. It relies on high-frequency hardware timers to accurately output SPWM variable frequency logic signals to command the work of the power layer. Additionally, acting as a shock absorber, it utilizes the built-in trapezoidal acceleration limit and sensorless soft-start algorithm to eliminate the switching jitter of the mechanical system.

2.3 Execution and Power Layer

This layer is composed of a three-phase brushless motor driver board, dedicated two-axis gimbal brushless

motors, and a 12V high-power DC power supply system, serving as the ultimate energy conversion and mechanical output terminal in the entire architecture. Its core function is to receive microvolt-level logic PWM signals from the drive layer, rely on power for current amplification, and convert them into strong alternating phase currents sufficient to drive the rotation of the magnetic field. This layer directly combats the physical gravity of the gimbal camera, structural friction, and physical inertia brought about by rapid movement, providing continuous holding torque and smooth traction to achieve stable following of the picture.

3. Communication Bus and Protocol Design

3.1 Custom Data Frame Structure

In the high-frequency asynchronous collaboration between the vision end and the motor control end, the efficiency and anti-interference ability of the communication bus directly determine the response delay of the gimbal. Since the system sets a communication baud rate of 115200 bps, if traditional ASCII plain text strings are used to transmit speed commands, it will not only occupy a large bus bandwidth but also consume significant character conversion computing power of the microcontroller, and it is extremely prone to parsing misalignment caused by packet loss. For this reason, the system specifically designed a lightweight, high-reliability binary data frame protocol with a fixed length of 14 bytes. This protocol operates data directly at the underlying memory level, and the physical layout of the data frame strictly follows the following six consecutive logic blocks:

First block: Dual-byte frame synchronization header (0x55 and 0xAA). The system stipulates that all legal data frames must use these two specific hexadecimal bytes as the starting flag. Only when these two bytes are monitored continuously will the underlying parsing engine start subsequent reception, completely isolating the interference of random electromagnetic noise on the bus.

Second block: Command control bit (1 byte). Used to identify the business logic attribute of the current data packet. For example, when this byte is assigned a specific value (such as 0x01), it clearly represents that the current payload content is gimbal dual-axis target speed setting, which reserves ample expansion space for the system's future functional iterations.

Third block: Payload length bit (1 byte). Accurately indicates the total number of physical bytes of the subsequent valid data payload with a hexadecimal value. In this communication cycle, the system needs to transmit two sets of single-precision floating-point numbers simultaneously, so this bit value is strictly locked to 0x08 (i.e., 8 bytes).

Fourth block: Core data payload (8 bytes). This is the communication carrier core of the entire protocol. It contains two sets of IEEE 754 standard floating-point numbers occupying 4 bytes each, which respectively map the expected angular velocities of the gimbal Pitch elevation axis and Yaw yaw axis.

Fifth block: Security checksum (1 byte). In a high-intensity electromagnetic environment with frequent motor

commutation and sudden large current changes, bus data is extremely prone to occasional bit inversions. This checksum byte is generated by all bytes of the command bit, length bit, and core data payload through a specific cumulative error prevention algorithm. After Pico receives the data, it will execute reverse verification. Once the comparison fails, it will directly discard the entire frame of data, fundamentally locking the risk of motor runaway and loss of control caused by garbled commands.

Sixth block: Frame end trailer (1 byte). Fixedly adopting 0x0D (hexadecimal carriage return), when the underlying hardware receives this byte and the verification is correct, the protocol frame is officially unpacked, and the latest target speed variable is safely handed over to the back-end soft start and acceleration limiter.

The specific communication protocol document is placed in the docs folder of the Github repository.

3.2 High Reliability Parsing and Exception Handling

3.2.1 State Machine Parsing

Under the high baud rate of 115200, the data issued by the vision end pours into the microcontroller in the form of a continuous and dense byte stream. To this end, this system abandons the fragile linear reception mechanism and introduces a rigorous finite state machine model to take over the byte-by-byte parsing work.

This state machine strictly defines multiple discrete states such as finding the frame header, reading the command, receiving the payload, and checksum comparison through the underlying Switch-Case logic structure. Whenever the serial port hardware receives a new byte, the state machine will be awakened and judge the legitimacy according to its current logic node. For example, in the initial state, the system remains silent to any byte other than 0x55; after receiving the core payload, the system will strictly compare the locally generated checksum with the checksum byte sent by the sender. In any link of the parsing chain, as long as an unexpected error occurs (such as frame header misalignment, overlong payload, verification failure, or missing trailer), the state machine will immediately clear the semi-finished frame currently cached and force a reset to the initial listening state. This fusing mechanism ensures the safety of motor operation.

3.2.2 Hardware Interrupt Preemption

Under the single-core computing power limitation of the microcontroller, the system faces extremely severe CPU resource scheduling challenges. On the one hand, the underlying motor controller needs to rely on a 500Hz high-frequency timer to maintain uninterrupted rapid calculations and duty cycle updates; on the other hand, the UART serial port has tens of thousands of communication bytes arriving randomly every second. If the conventional code polling architecture is allowed to run, when the CPU is executing complex soft start or anti-shake mathematical calculations, the serial port bytes that happen to arrive cannot be extracted in time, directly causing serious overflow errors in the underlying hardware receiving buffer, resulting in gimbal tracking stuttering.

To solve the resource competition between communication and control, we elevated the priority of the UART

hardware receive interrupt to the highest level of the system. Under this architecture, no matter what complicated tasks the main loop is processing, as long as a valid level transition comes over the communication line, the UART hardware will interrupt all current calculations. The CPU will be forcibly suspended and switched to the serial port service environment to safely transfer this byte into the state machine cache, and then seamlessly resume the previous physical wave calculation. This prevents the hidden danger of high-frequency computing blocking communication on the premise of not affecting the continuity of motor wave generation at all.

4. Core Algorithms

4.1 Upper-Level Vision and Trajectory Planning (Raspberry Pi Node)

The upper-level system, deployed on the Raspberry Pi 5, serves as the central intelligence of the gimbal. It is responsible for real-time visual perception, human-robot interaction (HRI), and translating spatial data into smooth, executable kinematic commands. To guarantee high throughput and low latency, the software architecture utilizes a multi-threaded C++ environment, ensuring the perception pipeline and the control pipeline operate asynchronously without blocking each other.

4.1.1 Advanced Skeletal Perception Engine (YOLOv8-Pose)

Traditional visual tracking systems heavily rely on Bounding Box (BBox) detection. However, empirical testing revealed a critical "center-of-mass drift" paradox: as a subject approaches the camera, their lower body is cropped out of the frame, causing the geometric center of the bounding box to shift drastically upward. This false error signal forces the pitch axis to continuously tilt downward, resulting in severe target loss.

To definitively solve this, the perception engine was upgraded to utilize YOLOv8-Pose, optimized via the NCNN framework for bare-metal edge inference.

Keypoint Anchoring: Instead of tracking a volatile bounding box, the system extracts 17 anatomical keypoints and explicitly locks the gimbal's composition to the subject's Nose (Keypoint 0).

Dynamic Fallback Mechanism: To ensure absolute robustness against occlusions, if the nose's confidence score drops below a 0.5 threshold, the algorithm dynamically calculates the midpoint between the Left and Right Shoulders (Keypoints 5 and 6) and applies a geometric offset to estimate the head's position. This ensures the target anchor remains rock-solid regardless of the subject's distance, posture, or partial framing.

4.1.2 Non-Contact HRI State Machine (Gesture Control)

To elevate the system from a passive tracker to an interactive robotic operator, a zero-overhead gesture control layer was embedded within the vision thread. By analyzing the relative spatial coordinates of the subject's wrists (Keypoints 9 and 10) against their nose, the system detects a "raised hand" gesture.

To prevent false positives from natural body language, a temporal cooldown and verification state machine was implemented. The system requires the gesture to be held for a precise 17 frames (approximately 1.5

seconds at an 86ms system latency) to trigger a state toggle (Tracking Paused / Resumed). This provides a highly intuitive, hands-free operational switch for solo creators.

4.1.3 Kinematic Trajectory Planning and Error Mapping

The most significant engineering challenge lay in converting raw pixel errors into fluid, cinematic motor velocities. Directly feeding pixel differences to the motors inherently causes violent jerks and high-frequency oscillations. Our Trajectory Planning thread handles this through a sophisticated three-stage filtering pipeline:

1. Soft Dead-Zone Filtering:

A visual dead-zone of +/- 12 pixels was established around the target compositional center (X=160, Y=100). If the subject's micro-movements (e.g., breathing or shifting weight) fall within this zone, the active error is zeroed out. This prevents the gimbal from neurotically hunting for a mathematically perfect center, drastically reducing motor heat and power consumption.

2. Proportional Control (P-Controller) & Velocity Clamping:

Once the target exits the dead-zone, the active pixel error is multiplied by a tuned proportional gain (Kp = 0.005 for both Yaw and Pitch) to calculate the target velocity (V_target). To prevent mechanical runaway or hardware damage during sudden large tracking errors, the output is strictly clamped to a maximum safe velocity envelope of [-0.9, 0.9].

3. Symmetric Linear Ramping (The Anti-Stutter Mechanism):

Early iterations of the control loop suffered from severe "sawtooth" oscillations (stuttering) on the Yaw axis due to asymmetrical acceleration and deceleration rates. The breakthrough solution was the implementation of a Symmetric Step Smoothing algorithm.

Rather than allowing the current motor velocity (V_current) to instantly jump to the target velocity (V_target), the planner forces the velocity to ramp up or down by a rigidly fixed, symmetric step limit (SYMMETRIC_STEP = 0.015) per 10ms control cycle. The kinematic logic is defined as follows:

$$V_{current_{t+1}} = \begin{cases} \min(V_{current_t} + Step, V_{target}) & \text{if } V_{current_t} < V_{target} \\ \max(V_{current_t} - Step, V_{target}) & \text{if } V_{current_t} > V_{target} \end{cases}$$

This symmetric damping effectively acts as a mechanical low-pass filter. It completely absorbs the high-frequency noise generated by YOLO pixel jitter and ensures that both acceleration and deceleration phases are executed with cinematic, robotic fluidity, representing the proudest control-engineering achievement of the

system.

4.2 Underlying Hard Real-time SPWM Wave Generation Engine

In the open-loop FOC control of brushless motors, generating smooth SPWM waveforms requires extremely frequent high-frequency trigonometric function calculations for the electrical angle. To completely break this physical computing power bottleneck, this system adopts a Look-Up Table (LUT) ultra-fast calculation architecture. During the system initialization stage, the microcontroller pre-calculates the discrete sine values for a complete electrical cycle and solidifies them in a continuous memory array. In the subsequent high-frequency interrupt wave generation, the system directly bypasses all trigonometric calculations and extremely quickly extracts the corresponding sine duty cycle parameters from memory through modulo and index mapping of the current electrical angle. This underlying reconstruction compresses the calculation time of a single wave generation to an extremely low level. While perfectly ensuring the continuity of high-frequency wave generation, it leaves extremely ample computing power redundancy for the high-priority parsing of the UART protocol.

4.3 Physical Anti-shake and Flexible Actuator Design

Under the motor open-loop control framework, the moments of motor starting and commutation are dangerous nodes where the entire system is extremely prone to mechanical damage and violent jitter. Its macroscopic manifestation is that the gimbal violently twitches accompanied by a dull physical collision sound during each power-on start. To mitigate this impact, this system introduces a Soft-Start algorithm at the underlying layer of the actuator. In the initial stage when the system takes over the motor, the wave generation engine does not immediately respond to the speed commands issued by the upper computer, but returns the output magnetic field rotation frequency to zero and locks it at a fixed initial electrical angle. At the same time, the system introduces a precise time differentiation mechanism in the voltage control dimension, causing the phase voltage output to the motor to rise from 0V extremely smoothly and linearly up to the set 4.5V rated voltage upper limit. Through this "voltage slow-release" underlying logic, the system realizes a silent and smooth start, which will greatly extend the physical lifespan of the gimbal bearings and motor rotors.

5. Troubleshooting & Optimization

5.1 Incompatibility Issue of the SimpleFOC Library During Testing

Phenomenon: The team originally planned to introduce a position encoder to monitor the physical absolute angle of the motor rotor in real-time and feed it back to the main control node, thereby achieving high-precision closed-loop FOC control. However, during the actual hardware joint commissioning process, despite the team conducting repeated physical wiring troubleshooting and code logic verification, once the closed-loop control code of the massive third-party open-source library was integrated, the brushless motor remained in a locked state or unable to respond to commands, and the underlying main loop of the system stalled.

Solution: Faced with the problem of the underlying logic black box being untestable, the team shelved the

complicated upper-layer encapsulation of the third-party library, extracted only its core SPWM three-phase voltage space vector calculation formula, and hard-coded it directly into the pure C++ underlying test logic to drive the motor independently. Test results showed that the motor could respond perfectly and rotate smoothly.

Based on this, the team identified the root cause of the problem: because the third-party open-source library was deeply bound to a specific high-level hardware abstraction layer in its design, it resulted in the inability to transmit the correct three-phase voltage information to the motor, thus preventing smooth control.

To ensure the high reliability and hard real-time control requirements of the system, the team decisively abandoned the completely referenced third-party closed-loop ecosystem, choosing to retain only its highly valuable three-phase voltage calculation and pin configuration functions. The overall system architecture was consequently reconstructed from a sensor closed-loop to an open-loop mode, clearing the underlying obstacles for the subsequent docking with the upper computer vision data stream.

5.2 Visual Perception and Closed-Loop Coupling: Challenges and Solutions

5.2.1 Phenomenon: The Compositional Drift Paradox

During dynamic tracking tests, a persistent and critical issue termed "Compositional Drift" was observed. When a subject walked towards the camera, decreasing the physical distance to the lens, the gimbal would erroneously pitch downward, eventually cropping the subject's head out of the frame and losing the target.

This occurred because the initial perception engine relied on a traditional Bounding Box (BBox) algorithm, which calculates tracking error based on the geometric center of the box. As the subject moved closer, their lower body inevitably fell outside the camera's field of view (FOV). This sudden truncation caused the bottom edge of the bounding box to shift upward, which in turn drastically elevated the computed geometric center. The trajectory planner interpreted this artificial center shift as the subject moving higher in the frame, and consequently commanded the pitch motor to tilt downwards to compensate, creating a paradoxical and failing tracking loop.

5.2.2 Solution: Anatomical Keypoint Anchoring via Pose Estimation

To definitively resolve this geometric flaw, the perception architecture was fundamentally upgraded from BBox tracking to Skeletal Keypoint Estimation using YOLOv8-Pose.

Instead of tracking an unstable geometric center susceptible to framing crops, the system was re-engineered to lock onto an absolute anatomical anchor—specifically, the subject's Nose (Keypoint 0). Because the anatomical position of a subject's nose remains constant relative to their body regardless of how much of their lower torso is cropped by the camera frame, the calculated visual error became absolute and immune to distance-induced framing changes.

Furthermore, a dynamic fallback mechanism was implemented: if the nose keypoint confidence drops below the 0.5 threshold due to occlusion or profile turning, the system dynamically calculates the midpoint between the left and right shoulders (Keypoints 5 and 6) and applies a geometric vertical offset to estimate the head's

position. This transition to keypoint anchoring completely eradicated compositional drift, ensuring the gimbal maintains a rock-solid, cinematic vertical composition regardless of the subject's proximity or partial occlusion.

6. Testing Results and Performance Evaluation

6.1 System Response Latency

A critical metric for any dynamic tracking gimbal is the end-to-end system latency—defined as the time elapsed from a subject moving in the physical world to the motors initiating a corrective rotation.

During our physical testing phase, the primary computational bottleneck was identified as the visual perception thread. By deploying the lightweight YOLOv8-Pose model optimized through the NCNN framework on the Raspberry Pi 5, the system achieved a highly stable single-frame processing time of approximately 86 milliseconds. When combined with the asynchronous control loop and UART transmission scheduling, the total end-to-end response latency was maintained at just over one-tenth of a second.

This near real-time performance proved highly effective during dynamic field tests. Even when the subject initiated sudden, unpredictable lateral movements or rapid depth changes, the tracking latency was low enough to guarantee that the anatomical anchor (the subject's nose) never escaped the camera's field of view before the mechanical compensation began.

6.2 Tracking Stability and Thermal Performance

Tracking stability was rigorously evaluated under extreme conditions, specifically focusing on the system's behavior during high-frequency directional switching and rapid target acceleration.

-Thermal Performance Under Stress:

In traditional servo or brushless gimbal systems, high-frequency pixel jitter from the AI vision node often forces the motors into a state of constant, microscopic vibration. This erratic hunting behavior typically leads to rapid motor heating and eventual thermal shutdown. However, our testing confirmed that the motor temperature remained remarkably stable, exhibiting only a negligible rise even after prolonged periods of active tracking. This excellent thermal efficiency is a direct result of the visual dead-zone architecture. By ignoring minute, inconsequential target shifts within the central frame, the motors are allowed to rest, completely eliminating unnecessary power draw and mechanical fatigue.

-Overshoot and Kinematic Fluidity:

Another critical test involved observing the gimbal's response to a fast-moving target that suddenly stops. Systems relying purely on raw proportional error frequently suffer from mechanical overshoot, where the camera swings past the target and has to reverse back, creating a pendulum effect.

```
sor format: 1536x864-SBGGR10_1X10/RAW - Selected CFE format: 1536x864-PC1B/RAW
[Vision] Hardware Capture Stream Started! Resolution: 640x480
[Vision] NAKED Mode Active. Tracking algorithm: Greedy Matching.
[PROFILER] [Control] Exec Time: 0.002 ms | Loop strictly at 100Hz
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Vision] Pipeline Latency: 86.4 ms/frame | Target Found: YES
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Vision] Pipeline Latency: 84.8 ms/frame | Target Found: YES
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Vision] Pipeline Latency: 86.0 ms/frame | Target Found: YES
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Vision] Pipeline Latency: 85.3 ms/frame | Target Found: YES
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Vision] Pipeline Latency: 85.9 ms/frame | Target Found: YES
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Vision] Pipeline Latency: 85.1 ms/frame | Target Found: YES
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Vision] Pipeline Latency: 86.8 ms/frame | Target Found: YES
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Vision] Pipeline Latency: 86.9 ms/frame | Target Found: YES
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Vision] Pipeline Latency: 85.1 ms/frame | Target Found: YES
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Control] Exec Time: 0.002 ms | Loop strictly at 100Hz
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Vision] Pipeline Latency: 85.9 ms/frame | Target Found: YES
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Vision] Pipeline Latency: 84.4 ms/frame | Target Found: YES
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
[PROFILER] [Control] Exec Time: 0.003 ms | Loop strictly at 100Hz
```

During our testing, the gimbal exhibited zero visible overshoot. This success validates the implementation of the Symmetric Step Smoothing algorithm within the trajectory planner. Because the acceleration and deceleration phases are strictly governed by a symmetric ramping limit, the gimbal smoothly and naturally tapers its velocity as the visual error approaches the dead-zone. The resulting movement perfectly mimics the fluid, dampened motion of a professional human camera operator, confirming the system's high cinematic reliability.

7. Conclusion and Future Work

7.1 Conclusion

This project successfully culminated in the engineering of a highly responsive, autonomous dual-axis vision-tracking gimbal, purpose-built for dynamic live streaming and solo content creation. By leveraging edge computing on the Raspberry Pi 5, the system achieves a highly stable, low-latency processing loop of approximately 86 milliseconds.

The system distinguishes itself through several key architectural breakthroughs. The integration of the YOLOv8-Pose model for anatomical keypoint anchoring effectively eradicated the compositional drift paradox inherent in traditional bounding-box tracking. Furthermore, the custom trajectory planner, featuring the Symmetric Step Smoothing algorithm, successfully translated raw visual pixel errors into fluid, cinematic motor actuations, completely eliminating mechanical stutter and high-frequency oscillations. The addition of a zero-overhead, temporal-filtered gesture control state machine elevated the gimbal from a passive tracker to an interactive robotic operator. Ultimately, the decisive architectural pivot to a streamlined, bare-metal open-loop SPWM drive ensured that the strict real-time constraints were met without stalling the system, validating the gimbal's robustness and operational reliability.

7.2 Future Work

While the current open-loop kinematic architecture exhibits exceptional stability and cinematic smoothness, it inherently operates at a continuous power draw to maintain motor torque, regardless of the physical load.

The primary objective for the next hardware iteration of this project is the implementation of true closed-loop Field-Oriented Control (FOC). This will be achieved by integrating AS5600 high-precision magnetic absolute encoders directly into the pitch and yaw motor assemblies. By feeding real-time, absolute rotor position data back to the lower-level microcontroller, the system will be capable of dynamically calculating the exact optimal vector required to align the magnetic fields. This advanced architectural upgrade will yield three major benefits: it will drastically reduce overall thermal output and power consumption, significantly increase torque efficiency to resist sudden external physical disturbances, and further elevate the micro-stepping precision of the gimbal's movements.

Additionally, future software iterations could explore the integration of multi-target tracking algorithms. By incorporating advanced ID persistence logic (such as ByteTrack), the gimbal could be programmed to memorize specific user IDs, allowing it to intelligently ignore crossing pedestrians and confidently maintain focus on the primary subject in highly complex, crowded environments.